

[Low today / High on next position migration] Ungated `save-pending-rewards` lets anyone permanently brick a holder's reward claims on a deactivated `stSTXbtc` position

Program: StackingDAO — <https://immunefi.com/bug-bounty/stackingdao/scope/>

In-scope assets: `SP4SZE494VC2YC5JYG7AYFQ44F5Q4PYV7DVMDPBG.ststxbtc-tracking-v2` and `.ststxbtc-tracking-data-v2` (Primacy of Impact; both are active protocol contracts and the live reward tracker for `stSTXbtc`)

Severity (self-assessed): Low at the time of writing — only dust sits in deactivated positions. It becomes High ("permanent freezing of unclaimed yield") the moment the DAO deactivates a position with material holdings, which is exactly what happened to `position-zest-v3/v4/v5` and what will happen to `position-zest-v6` (7,988,255,302,396 tracked units) on its next migration. Reported now so the fix lands before that.

Funds at risk today: a few sats across three deactivated positions. Potential: all unclaimed sBTC rewards of every holder of the next deactivated position.

Summary

`ststxbtc-tracking-v2.save-pending-rewards(holder, position)` is `define-public` with no caller check. For a deactivated position it computes the holder's pending rewards against the frozen checkpoint `deactivated-cumm-reward` (D), saves them, and then resets the holder's checkpoint to the current global `cumm-reward` (G), which is greater than D. Every subsequent `get-pending-rewards(holder, position)` evaluates `(- D holder.cumm)` with `holder.cumm = G > D`, which is a uint underflow — a Clarity runtime abort. `claim-pending-rewards` `unwrap-panics` on that value, so the holder's saved rewards become unclaimable. Anyone can do this to any holder with one transaction.

Vulnerability details

`ststxbtc-tracking-v2.clar`:

```
;; lines 82-97 — public, ungated
(define-public (save-pending-rewards (holder principal) (position principal))
  (let ((pending-rewards (unwrap-panic (get-pending-rewards holder position)))
        (existing-rewards (get-saved-rewards holder position)))
    (if (> (- pending-rewards existing-rewards) u0)
        (begin
```

```

    (map-set saved-rewards { holder: holder, position: position } pending-rewards)
    (try! (contract-call? .ststxbtc-tracking-data-v2 update-holder-position holder position
      (ok pending-rewards))
      (ok u0))))

;; lines 120-145
(define-read-only (get-pending-rewards (holder principal) (position principal))
  (let (...
    (cumm-reward (if (and (not (is-eq holder position))
      (not (is-eq (get deactivated-cumm-reward supported-position) u0)))
      (get deactivated-cumm-reward supported-position) ;; D, frozen at deactivation
      (contract-call? .ststxbtc-tracking-data-v2 get-cumm-reward)))
    (amount-owed-per-token (- cumm-reward (get cumm-reward holders-info)) ;; D - G underflows
    ...

```

`ststxbtc-tracking-data-v2.update-holder-position` is protocol-gated on `contract-caller`, but the caller is `ststxbtc-tracking-v2`, an active protocol contract, so the inner call succeeds for any external caller of `save-pending-rewards`. Deactivation (`set-supported-positions ... false`, lines 215–218) stores `deactivated-cumm-reward = G_at_deactivation` precisely so holders can still claim what they earned up to that point; the bug defeats that guarantee.

The holder cannot recover on their own: `refresh-position` asserts the position is active, and `claim-pending-rewards` aborts. Only a DAO call to `ststxbtc-tracking-data-v2.set-holder-position(holder, position, amount, D)` repairs each affected holder individually.

Live state

position	active	deactivated-cumm-reward (D)	tracked total
position-zest-v3	false	188,092	2,210,000
position-zest-v4	false	188,092	1,705,000
position-zest-v5	false	194,129	500,000
position-zest-v6	true	—	7,988,255,302,396

Global `cumm-reward` = 293,987 > D for all three deactivated positions, so the underflow is reachable today for any of their holders who has not yet claimed. The included read-only verifier `verify_tracking_griefing.py` reproduces this table live from the Hiro API (no wallet, no transaction).

Steps to reproduce (Clarinet simnet; no mainnet interaction)

- Deploy the tracker, data contract, token and a minimal position adapter; activate the position; give Alice a position balance and `refresh-position`.
- `add-rewards` so Alice accrues; deactivate the position (`set-supported-positions ... false`); `add-rewards` again so global cumm exceeds D.
- `get-pending-rewards(alice, position)` returns her earned amount. `claim-pending-rewards` would succeed here.
- From any unrelated account call `save-pending-rewards(alice, position)`. It returns `(ok <alice's pending>)`.
- `get-pending-rewards(alice, position)` now aborts with an arithmetic underflow; `claim-pending-rewards(alice, position)` aborts. Alice's saved rewards are frozen until the DAO repairs her checkpoint.

Impact

Griefing with no attacker profit today (dust at risk). On the next deactivation of a live position it becomes a one-transaction-per-victim permanent freeze of unclaimed sBTC rewards, recoverable only by a per-holder governance intervention. The pattern also affects `refresh-position`, which calls `save-pending-rewards` first and therefore also aborts for these holders.

Recommended fix

- Gate `save-pending-rewards` with `(try! (contract-call? .dao check-is-protocol contract-caller))` — it is only ever called internally.
- Independently, make the checkpoint reset safe for deactivated positions: in `save-pending-rewards` (or in `update-holder-position`) set the holder's cumm to `min(G, D)` when `D != 0`, or short-circuit `get-pending-rewards` to return only `saved-rewards` when `holder.cumm >= D`.
- Repair `position-zest-v3/v4/v5` holders whose checkpoints already exceed D, if any.

Reward

Delivered privately to the StackingDAO team as a companion to the stBTC reward-split report (report 01), at Low today with a documented path to High. Requested for consideration under the team's discretionary policy; the fix is two lines and prevents a real freeze on the next position migration.